POLYBOT

Product Requirements Document

Autonomous Polymarket Trading System

Version	v1.0
Date	March 2025
Status	Ready for Claude Code
Author	Polybot Team

© 2025 Polybot. All rights reserved.

1. Executive Summary

Polybot is a fully autonomous, AI-powered prediction market trading engine that operates on Polymarket (Polygon/USDC). The system deploys multiple specialized Claude AI agents that continuously scan live markets, identify statistically significant mispricing, and execute real-money trades — targeting small, consistent daily gains with strict risk controls.

This document defines the complete product requirements for the Polybot system, intended as the authoritative specification for implementation via Claude Code.

2. Problem Statement

2.1 Market Opportunity

Prediction markets such as Polymarket consistently exhibit mispricing due to:

- Crowd sentiment bias — markets over-react to recent news
- Thin liquidity in niche categories allowing edges to persist
- Lack of systematic, data-driven participation from retail traders
- Lag between real-world signal availability and market price adjustment

2.2 Current Gap

Manual trading is slow, emotional, and inconsistent. No open-source autonomous agent exists that combines real-time news ingestion, AI probability estimation, risk-managed order execution, and a live monitoring dashboard in a single deployable system.

3. Product Goals

3.1 Primary Objectives

- Generate consistent small daily gains on Polymarket ($5–$30/day target)
- Operate fully autonomously with zero manual intervention required
- Maintain strict daily loss limits to protect capital
- Surface full transparency via a real-time web dashboard

3.2 Non-Goals

- High-frequency trading or sub-second execution
- Support for exchanges other than Polymarket in v1
- Portfolio management across non-prediction-market assets
- Mobile application

© 2025 Polybot. All rights reserved.

4. Target Users

User	Description	Primary Need
Operator	Runs the system on their own server/machine	Reliable autonomous trading with low oversight
Analyst	Reviews trade decisions and agent reasoning	Full trade log with Claude reasoning visible
Trader (manual)	Injects specific markets via dashboard override	Manual trade + AI analysis on demand

5. Functional Requirements

5.1 Agent System

REQ-AGT-001

The system shall deploy a minimum of 5 specialized agents, each owning a distinct market category: Crypto, Politics, Economics, Sports, Weather, and General (high-volume misc).

REQ-AGT-002

Each agent shall scan its target markets on an independent configurable interval (30–120 seconds) with randomized stagger on startup to avoid API rate limit collisions.

REQ-AGT-003

Each agent shall analyze a maximum of 3 markets per scan cycle to control Claude API costs and latency.

REQ-AGT-004

Agents shall be independently startable and stoppable without affecting other agents.

5.2 Market Analysis

REQ-ANA-001

Before each analysis, agents shall fetch real-time contextual data: crypto news (CryptoCompare), weather data (Open-Meteo), and general headlines (GNews). This context shall be injected into the Claude prompt.

REQ-ANA-002

Claude shall return a structured JSON decision containing: trade direction (YES/NO/SKIP), true probability estimate, confidence score, edge percentage, risk level (LOW/MEDIUM/HIGH), reason, key factor, and warning.

REQ-ANA-003

The system shall filter Polymarket markets by minimum 24h volume ($100) and minimum liquidity ($200) before passing to agents.

5.3 Trade Execution

REQ-EXE-001

© 2025 Polybot. All rights reserved.

The system shall integrate with the Polymarket CLOB API for order placement. All orders shall be placed as Fill-Or-Kill (FOK) to avoid partial fills.

REQ-EXE-002

Stake per trade shall be dynamically calculated using a fractional Kelly formula (0.25x Kelly), bounded between MIN_STAKE_USD and MAX_STAKE_USD as configured in the environment.

REQ-EXE-003

All order activity shall be logged with: market question, trade direction, stake, edge, confidence, Claude reasoning, order ID, and timestamp.

REQ-EXE-004

If no wallet key is configured, the system shall operate in READ-ONLY simulation mode — all analysis runs but no orders are placed.

5.4 Risk Management

REQ-RSK-001

A central RiskManager module shall be the single authority for trade approval. All agents must pass through RiskManager.approve() before any order is placed.

REQ-RSK-002

The following conditions must ALL be true for a trade to be approved: |edge| >= MIN_EDGE_PCT, confidence >= MIN_CONFIDENCE_PCT, open trade count < MAX_OPEN_TRADES, daily P&L > -DAILY_LOSS_LIMIT, engine not halted, risk_level != HIGH unless |edge| >= 10%.

REQ-RSK-003

When the daily loss limit is breached, the engine shall halt all agents immediately and emit a HALTED state to the dashboard. Manual resume is required via the dashboard or API call.

REQ-RSK-004

Daily P&L shall reset automatically at midnight UTC via a scheduled cron job.

5.5 Dashboard

REQ-DSH-001

A web dashboard shall be accessible at http://localhost:3000. It shall display: total P&L, daily P&L, win rate, open trade count, scans completed, daily loss limit usage, live P&L curve, agent status panel, full trade log, and news/signal feed.

REQ-DSH-002

Dashboard data shall update in real time via WebSocket (port 3001) — no page refresh required.

REQ-DSH-003

The dashboard shall include controls to: stop the engine, start the engine, reset daily P&L, and submit manual trade overrides (market question, category, probability, stake).

REQ-DSH-004

The dashboard shall display wallet address (truncated) and live USDC balance, refreshed every 5 minutes.

© 2025 Polybot. All rights reserved.

6. Non-Functional Requirements

6.1 Performance

- Agent scan-to-decision latency: < 8 seconds (including Claude API call)
- WebSocket state broadcast: < 200ms after state change
- Dashboard initial load: < 2 seconds on localhost

6.2 Reliability

- Each agent scan shall be wrapped in try/catch — agent errors must not crash the engine
- WebSocket disconnects shall auto-reconnect with 3-second backoff
- API failures (Polymarket, news, Claude) shall be logged and skipped gracefully

6.3 Security

- Wallet private key and API credentials shall be loaded exclusively from .env — never hardcoded
- .env shall be listed in .gitignore — the repo must never contain real credentials
- Dashboard shall be localhost-only by default — no external exposure

6.4 Observability

- Winston logger writes to: console (colored), logs/trades.log, logs/error.log
- All trade opens, closes, skips, and agent errors shall be logged at INFO level
- Log files shall rotate at 5MB / 10 files maximum

7. Configuration

All runtime parameters shall be configurable via environment variables with sensible defaults:

Variable	Default	Description
MAX_STAKE_USD	20	Maximum USDC stake per trade
MIN_STAKE_USD	1	Minimum USDC stake per trade
DAILY_LOSS_LIMIT_USD	50	Engine halts if daily loss exceeds this
MAX_OPEN_TRADES	8	Maximum simultaneous open positions
MIN_EDGE_PCT	5	Minimum edge % required to trade
MIN_CONFIDENCE_PCT	62	Minimum Claude confidence % to trade
SCAN_INTERVAL_SECONDS	45	Base scan interval per agent

© 2025 Polybot. All rights reserved.

8. Acceptance Criteria

8.1 Engine

1. All 6 agents start successfully on node index.js
2. Agents scan independently and do not block each other
3. Trades are approved only when all RiskManager conditions pass
4. Daily loss limit halts the engine within 1 scan cycle of breach
5. Engine resumes after manual dashboard action

8.2 API Integration

6. Polymarket CLOB API connects and returns active markets
7. Markets are filtered by volume and liquidity before analysis
8. Orders are placed via CLOB with correct wallet signature
9. READ-ONLY mode works correctly with no wallet key

8.3 Dashboard

10. Dashboard loads at http://localhost:3000
11. WebSocket updates P&L and trade log in real time
12. Stop/Start engine buttons immediately affect agent state
13. Manual trade override runs full Claude analysis and executes if approved

9. Out of Scope — v1

- Kalshi integration
- Mobile app or native desktop app
- Automated backtesting framework
- Multi-wallet support
- Social / copy-trading features
- Email / SMS alerts

© 2025 Polybot. All rights reserved.